Course Title: ARTIFICIAL NEURAL NETWORKS AND DEEP LEARNING (V sem) - UG	Course Code: AI253IA – Test II (S&S)

Sl No	Questions	M	BT	CO
1.	Examine the effectiveness of the ReLU layer compared to other activation functions (sigmoid and Tanh) in CNNs. 1) Effectiveness of ReLU vs Sigmoid and Tanh in CNNs ReLU ( ) • Pros: fast to compute, encourages sparse activations (many zeros), and reduces vanishing gradient for positive inputs → usually trains deeper CNNs better/faster. • Cons: "dying ReLU" problem (neurons can get stuck outputting 0 if inputs become negative consistently). Sigmoid ( ) • Pros: outputs in (0,1) → good for probability/logistic outputs. • Cons in CNN hidden layers: saturates for large ± inputs → vanishing gradients, slower learning, outputs not zero-centered. Tanh ( ) • Pros: outputs in (-1,1) and is zero-centered, often better than sigmoid for hidden layers. • Cons: still saturates → vanishing gradients (though typically less severe than sigmoid). Conclusion: In CNN hidden layers, ReLU is generally more effective (faster convergence + better gradient flow). Sigmoid/tanh are more commonly used in output **layers** or in gating mechanisms (e.g., LSTM/GRU).	**2**	**4**	**1**
2.	Given the values for input gate (I_t=0.8), forget gate (f_t=0.6), output gate (O_t=0.7), previous cell state (C_{t-1}=0.5), and candidate cell state (C_t=4), calculate the updated cell state C_t using the LSTM cell state update equation **LSTM updated cell state** **Given:** **LSTM cell update equation:** Substitute: **Updated cell state:**	**2**	**2**	**1**
3.	Given x_t=2.5,h_{t-1}=0.9, W_{xh}=0.4,W_{hh}=0.8, and b_h=0.5 calculate the hidden state h_t at time step t using the RNN update equation **RNN update equation (with tanh):** Compute inside: Now: **Hidden state:**	**2**	**2**	**1**
4.	List any two differences between LSTM and GRU **Any two differences between LSTM and GRU** 1. **Gates** • **LSTM:** 3 gates (input, forget, output) + separate cell state • **GRU:** 2 gates (update, reset) and no separate cell state 2. **Complexity** • **LSTM:** more parameters → heavier/slower but strong long-term memory control • **GRU:** fewer parameters → typically fast**er**, often similar performance	**2**	**1**	**1**
5.	Perform max pooling on the given 4×4 feature map using a 2×2 window size with stride 2. Show the resulting pooled output. 3 1 1 3 2 5 0 2 1 4 2 1 4 7 2 4	**2**	**3**	**1**

	Window size , stride 2 → pooled output is . **Block 1 (rows 1–2, cols 1–2):** **Block 2 (rows 1–2, cols 3–4):** **Block 3 (rows 3–4, cols 1–2):** **Block 4 (rows 3–4, cols 3–4):** **Pooled output:**			

Part B

Sl No	Questions	M	BT	CO
2a.	How is backpropagation implemented through **matrix multiplication** in Convolutional Neural Networks (CNNs)?	5	3	1
2b.	Assume you are tasked with classifying medical images for the detection of lung cancer, but the dataset consists of a limited number of images. Identify and justify the technique you would use to increase the training set and explain the process in detail. **1. Technique to Increase Training Set for Lung Cancer Image Classification** **Identified Technique: Data Augmentation (with Transfer Learning support)** When medical image datasets (e.g., CT scans, X-rays) are limited, data augmentation is the most effective and widely used technique to artificially increase the size and diversity of the training set without collecting new data. **Justification** • Medical image collection is expensive, time-consuming, and ethically constrained • CNNs require large datasets to avoid overfitting • Augmentation improves generalization and robustness • Preserves original labels while creating new valid samples **Augmentation Techniques Used** Let original image be 1. **Geometric Transformations** ○ Rotation: ○ $I'(x,y)=I(x\cos\theta-y\sin\theta,\ x\sin\theta+y\cos\theta)$ ○ Horizontal/Vertical flipping ○ Scaling and cropping 2. **Intensity Transformations** ○ Contrast adjustment ○ Brightness normalization ○ Gaussian noise addition 3. **Elastic Deformation** ○ Useful for medical images to simulate anatomical variability 4. **Advanced Methods** ○ GAN-based synthetic image generation ○ MixUp and CutMix **Process Flow** 1. Start with limited labeled lung images 2. Apply random transformations during training 3. Generate multiple variants per image 4. Train CNN on augmented dataset 5. Optionally combine **with transfer learning** (e.g., ResNet, VGG) **Outcome:** Increased dataset size, reduced overfitting, improved cancer detection accuracy.	5	4	1
3a.	Provide a detailed explanation of the mathematical formulations for Bidirectional and Multilayer Recurrent Neural Networks (RNNs), including	5	3	3

	the state transition equations for both forward and backward passes in BRNNs.
	
	A. Standard RNN
	Given input , hidden state :
	
	B. Bidirectional Recurrent Neural Network (BRNN)
	BRNN processes sequences in **both forward and backward directions**, capturing past and future context.
	Forward Pass
	B. Bidirectional Recurrent Neural Network (BRNN)
	BRNN processes sequences in **both forward and backward directions**, capturing past and future context.
	Forward Pass
	$$\overrightarrow{h_t} = \phi(W_{x\rightarrow}x_t + W_{\rightarrow}\overrightarrow{h_{t-1}} + b_{\rightarrow})$$
	Backward Pass
	$$\overleftarrow{h_t} = \phi(W_{x\leftarrow}x_t + W_{\leftarrow}\overleftarrow{h_{t+1}} + b_{\leftarrow})$$
	Output Combination
	$$h_t = [\overrightarrow{h_t};\ \overleftarrow{h_t}]$$
	Final Output
	$$y_t = W_y h_t + b_y$$
	C. Multilayer (Stacked) RNN
	In a multilayer RNN, the output of one RNN layer serves as input to the next.
	• When an Input is passed to Layer 1:
	  • The input ((passes through the RNN layer 1. There, the hidden state gets updated as:
	
	  • where
	  • =present Hidden state, =previous hidden state
	  • =input to the RNN layer, =weights associated with the input, =weights associated with the hidden layer
	  • =bias associated with RNN layer, = activation function
	• The present hidden state is used in getting the output of the hidden layer, using an appropriate activation function.
	  • where,
	  • W = Weights assigned to the layer, =hidden state
	  • =bias associated with output layer
	  • For the second layer, the output of first RNN layer is fed into it, which goes through the same process again.			

3b	Provide a comprehensive technical overview of a Convolutional Neural Network (CNN) architecture. Please include a brief explanation of each layer, such as the input layer, convolutional layers, activation functions, pooling layers, fully connected layers, and the output layer. **1. Input Layer** • Accepts raw image • Example: **2. Convolutional Layer** Applies learnable filters to extract features. • Detects edges, textures, shapes • Weight sharing reduces parameters	5	2	1

	3. Activation Function Introduces non-linearity. **ReLU (most common):** **4. Pooling Layer** Reduces spatial dimensions and computation. **Max Pooling:** • Provides translation invariance **5. Fully Connected Layer** • Flattens feature maps • Learns global feature combinations **6. Output Layer** • Produces final predictions • **Softmax** for multi-class:			
4a.	In a smart grid energy management system, an Echo State Network (ESN) is used to forecast energy demand based on real-time data from IoT sensors. The system receives input features such as current energy consumption, weather conditions, and time of day. The ESN is trained to predict energy demand for the next hour. **a)** Describe the architecture of the ESN in this scenario, highlighting the roles of the input, reservoir, and output layers. **(a) ESN Architecture Description** An ESN is a type of **Reservoir Computing** model where only the output layer is trained. **1. Input Layer** • Inputs: energy usage, weather, time • Weight matrix: **2. Reservoir (Dynamic Hidden Layer)** • Large, fixed, sparsely connected recurrent network • Captures temporal dynamics • Echo State Property ensures stability **3. Output Layer** • Only trainable part **Advantages** • Fast training • Suitable for real-time IoT data • Captures nonlinear temporal dependencies	5	3	3
4b.	Consider the statement, *"Head Coach GG fail in Home Test Series."* Discuss briefly how a Long Short-Term Memory (LSTM) network can be effectively utilized for grammatical error detection and correction in such sentences. Provide the architecture diagram of an LSTM cell and provide mathematical equations for the cell state **How LSTM Helps** • Captures long-term dependencies • Understands grammatical structure • Detects subject–verb agreement errors • Learns context-sensitive corrections	5	3	1

Example correction:

"Head Coach GG **failed** in the Home Test Series."

Mathematical Equations of LSTM Cell

Let , , :

Forget Gate:

Input Gate:

Candidate Cell State:

Cell State Update:

Output Gate:

Hidden State:

Why LSTM is Effective

- Avoids vanishing gradients
- Retains long-range syntax
- Used in grammar correction, machine translation, NLP tasks

5a.	Given the 6×6 input matrix and the 3×3 filter shown below, perform the following operations. Use **zero padding** and **stride = 2** for the convolution, treat the filter as a cross-correlation (no kernel flip), and show intermediate 3×3 blocks and calculations where helpful.	6	3	2

Input 6*6 matrix

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Filter 3*3

1	0	-1
1	0	-1
1	0	-1

a. Compute the 2D convolution (cross-correlation) output with zero padding, **stride = 2**. Show the 3×3 blocks used and the resulting output matrix.
b. Apply **ReLU activation** to the convolution output and show the result.
c. Perform **max pooling** on the ReLU output using a **2×2 window** with **stride = 2**. Show the pooling windows and the final pooled output.

A) Convolution output (cross-correlation), padding , stride

Step 1: Zero-pad the input (becomes 8×8)

block (rows 0–2, cols 0–2)

Left col sum , Right col sum ,
block (rows 0–2, cols 2–4)

Left sum , Right sum ,
block (rows 0–2, cols 4–6)

Left sum , Right sum ,

block (rows 2–4, cols 0–2)

Left sum , Right sum ,

block (rows 2–4, cols 2–4)

Left sum , Right sum ,

block (rows 2–4, cols 4–6)

Left sum , Right sum :

block (rows 4–6, cols 0–2)

Left sum , Right sum :

block (rows 4–6, cols 2–4)

Left sum , Right sum ,
block (rows 4–6, cols 4–6)

Left sum , Right sum ,
Final convolution (cross-correlation) output (3×3)

B) Apply ReLU

So all negatives become 0:

C) Max pooling (2×2 window, stride 2) on ReLU output

ReLU output is . With , stride 2, **only one full window fits** (top-left). **Pool window 1: rows 0–1, cols 0–1**

Final pooled output:

5b.

Analyze the different topologies of Recurrent Neural Networks (RNNs) and discuss how each topology is applied in real-world scenarios.

Recurrent Neural Networks (RNNs) are designed to process **sequential and temporal data** by maintaining hidden states that capture information from previous time steps. Different **RNN topologies** define how inputs, hidden states, and outputs are connected over time, enabling RNNs to solve various real-world sequence modeling problems.

- **One-to-One:** Represents vanilla neural network (CNN) of processing without recurrent nets, from constant-sized input to constant-sized output (e.g., image classification).
- **One-to-many:** A one-to-many neural network architecture refers to models where a single, fixed-size input is used to generate a sequence of outputs. (e.g., image captioning acquires an image as input and outputs a sentence of words).
- **Many-to-one:** A many-to-one neural network architecture processes a sequence of inputs and produces a single output. (e.g., sentiment analysis where a known sentence is classified as stating positive or negative sentiment).
- **Many-to-many for sequence input and sequence output :** A many-to-many neural network architecture handles sequence inputs and sequence outputs. A common example is language translation, where an RNN-based model (or a modern Transformer) reads a sentence in English word by word and then generates a corresponding sentence in French word by word. Both the input and output are variable-length sequences.
- **Many-to-many for synced sequence input and output:** Many-to-many for synced sequence input and output, e.g., video classification, where we want to label every frame.

4 | 4 | 3

6.

Consider the convolutional neural network defined by the layers in the left column below. Fill in the shape of the output volume and the number of parameters at each layer. You can write the shapes in the NumPy format (e.g. (128,128,3)) (Perform step by step calculations)

Notation:

a. CONV5-N denotes a convolutional layer with N filters with height and width equal to 5. Padding is 2, and stride is 1.
b. POOL2 denotes a 2x2 max-pooling layer with stride of 2 and 0 padding.
c. FC-N denotes a fully connected layer with N neurons.

Layer	**Activation Volume Dimensions**	**Number of parameters**
Input	**32*32*1**	**0**
CONV5-10		
POOL2		
CONV5-10		
POOL2		
FC10		

10 | 3 | 2

1) CONV5-10

-

Output spatial size

Depth , Output shape: **(32, 32, 10)**
Parameters
Each filter: weights
10 filters → weights
Biases: Params:

2) POOL2 (2×2, stride 2)
Input: (32, 32, 10)
Output spatial size

Depth unchanged = 10, Output shape: **(16, 16, 10),** Params: **0**

3) CONV5-10
Input: (16, 16, 10)

Output spatial size

Depth , Output shape: **(16, 16, 10)**
Parameters
Each filter: weights
10 filters → weights, Biases: 10, Params:

4) POOL2
Input: (16, 16, 10)
Output spatial size

Depth unchanged = 10, Output shape: **(8, 8, 10),** Params: **0**

5) FC10
Input volume: (8, 8, 10)
Flatten

Parameters

Params:
Output shape: **(10,)** (or simply 10 neurons)

Layer	**Activation Volume Dimensions**	**# Parameters**
Input	(32, 32, 1)	0
CONV5-10	(32, 32, 10)	260
POOL2	(16, 16, 10)	0
CONV5-10	(16, 16, 10)	2510
POOL2	(8, 8, 10)	0
FC10	(10,)	6410